



**Politecnico
di Torino**

this:

POLITECNICO DI TORINO

College of Computer Engineering, Cinema and
Mechatronics

Master's Degree Thesis

Insert title here

Supervisors

prof. Bartolomeo MONTRUCCHIO

dr. Antonio Costantino MARCEDDU

Candidate

Alessio TANZI

LUGLIO 2026

Abstract

Insert a brief summary of the thesis here.

Acknowledgements

Optional, only in case you have received special and particularly relevant help.

Contents

List of Figures	VI
List of Tables	VII
Acronyms	VIII
1 Scrivere testi complessi	1
1.1 Scrittura di un capitolo	1
1.2 Installazione di un sistema $\text{\LaTeX}$	1
1.3 Comporre testi con $\text{\LaTeX}$	1
1.3.1 Testo normale	1
1.3.2 Liste	3
1.3.3 Formule matematiche ed unità di misura	4
1.3.4 Tabelle e figure	5
1.4 La bibliografia	6
1.5 Acronimi	7
1.6 Conclusioni	8
2 La tesi di laurea magistrale	9
2.1 Struttura della tesi	9
2.2 Writing the thesis in English	9
3 Progettazione	10
4 Risultati	11
5 Conclusioni	12
6 Foundations	13
6.1 Numerical Methods for Differential Equations	13
6.1.1 Ordinary Differential Equations	13
6.1.2 Vector Spaces and Explicit Methods	14
6.1.3 Systems of ODEs and Higher-Order ODEs	14
6.1.4 Symplectic Integrators and Error Bounds	15

6.2	Rigid Body Dynamics	16
6.2.1	Translational Dynamics	16
6.2.2	Rotational Dynamics	16
6.2.3	Inertia Tensor Computation	18
6.3	Rotational Representations and Lie Groups	18
6.3.1	Introduction to Lie Groups	18
6.3.2	Lie Algebras and the Hat Operator	18
6.3.3	Numerical Integration on Manifolds	19
6.4	Implicit Integration Methods	19
6.4.1	Root-Finding and Newton-Raphson	19
6.4.2	Implicit Integration	21
6.5	Coupled Particle and Rigid Body Integration	22
6.5.1	Configuration Manifolds and Continuous Dynamics	22
6.5.2	The Staggered IMEX Integration Loop	22
6.5.3	Proof of Exact Time-Reversibility	23
6.5.4	In-Depth Analytical Jacobian on Lie Group $SO(3)$	24
	Bibliography	26

List of Figures

1.1	Popolazione di Torino, in migliaia di abitanti (fonte: wikipedia).	6
1.2	Esempio di programma inserito tramite <code>lstlisting</code>	6
1.3	Esempio di programma inserito tramite <code>lstlisting</code> con numerazione delle righe. .	7
6.1	Comparison of numerical integration methods. Explicit Euler unconditionally diverges for exponentially growing or stiff systems, while higher-order methods gracefully track the true continuous solution over short intervals.	14
6.2	Energy conservation in conservative systems. Symplectic methods exhibit strictly bounded oscillations tightly around the exact initial energy, elegantly avoiding the secular, non-physical drift characteristic of standard explicit methods over long durations.	15
6.3	Kinematics and dynamics of a rigid body. An external force $\mathbf{F}_{\text{ext}}$ applied at relative position $\mathbf{r}$ induces both a linear acceleration of the Center of Mass (CoM) and a torque $\boldsymbol{\tau}$ modifying the angular velocity $\boldsymbol{\omega}$	17
6.4	Lie Group Integration. Standard linear updates push the state off the curved manifold. Lie group methods utilize the local Lie algebra and the exponential map to seamlessly advance the state along the true geometric surface.	20
6.5	The Newton-Raphson method for root-finding. By iteratively evaluating the function and its local derivative (tangent line, dashed red), the sequence of estimates $y^{(i)}$ rapidly converges towards the true root y^*	21
6.6	Partitioned Implicit-Explicit (IMEX) timeline. The particle system is explicitly stepped using a split Velocity Verlet scheme, which perfectly synchronizes the configuration exactly at the midpoint $t_{n+1/2}$. This intermediate state is locked to evaluate the computationally expensive implicit rigid body Newton-Euler solver before advancing both systems to t_{n+1}	23

List of Tables

1.1	Voti riportati negli esami sinora sostenuti.	5
-----	--	---

Acronyms

This document is incomplete. The external file associated with the glossary ‘acronym’ (which should be called `main.acr`) hasn’t been created.

Check the contents of the file `main.acn`. If it’s empty, that means you haven’t indexed any of your entries in this glossary (using commands like `\gls` or `\glsadd`) so this list can’t be generated. If the file isn’t empty, the document build process hasn’t been completed.

Try one of the following:

- Add `automake` to your package option list when you load `glossaries-extra.sty`. For example:

```
\usepackage[automake]{glossaries-extra}
```

- Run the external (Lua) application:

```
makeglossaries-lite.lua "main"
```

- Run the external (Perl) application:

```
makeglossaries "main"
```

Then rerun $\text{\LaTeX}$ on this document.

This message will be removed once the problem has been fixed.

Chapter 1

Scrivere testi complessi

1.1 Scrittura di un capitolo

Dividere la tesi in capitoli (col comando `\chapter`) ed il capitolo in parti logiche mediante gli appositi comandi (`\section`, `\subsection` e `\subsubsection`).

Cercare inoltre di scrivere in buon italiano perché la tesi è un documento formale che viene archiviato per lungo tempo e costituisce parte della carriera di uno studente.

Questo capitolo di esempio contiene informazioni circa l'uso del sistema $\text{\LaTeX}$ per la scrittura di testi scientifici (molto utile anche per la composizione della tesi di laurea). Si consiglia di non limitarsi solo a leggere questo testo ma di esaminare il file `.tex` corrispondente per imparare rapidamente i comandi $\text{\LaTeX}$ tramite gli esempi contenuti in tale file sorgente.

1.2 Installazione di un sistema $\text{\LaTeX}$

Esistono tanti sistemi per elaborare testi scritti in $\text{\LaTeX}$. Per sistemi Windows si consiglia l'uso di MikTeX, che può essere scaricato dal seguente sito:

<http://www.miktex.org/>

MikTeX installa automaticamente anche TeXworks, un IDE (Integrated Development Environment) per $\text{\LaTeX}$ che permette di scrivere il sorgente ed ottenere velocemente il PDF corrispondente, che viene mostrato in una finestra separata. TeXworks è integrato con un correttore ortografico ed è dotato di auto-completamento delle parole (ad esempio scrivendo `bit` e premendo quindi TAB viene generato automaticamente l'ambiente `itemize` per inserire una lista non ordinata); si veda la sezione 6.3 del manuale di TeXworks per un elenco completo. Notare anche che facendo click col tasto destro nella finestra che visualizza il PDF è possibile saltare automaticamente al punto corrispondente nella finestra di edit e viceversa.

1.3 Comporre testi con $\text{\LaTeX}$

In questa sezione vengono fornite alcune informazioni generali sull'uso del linguaggio $\text{\LaTeX}$ per comporre testi complessi.

1.3.1 Testo normale

Le lettere accentate si possono scrivere direttamente (mediante i tasti presenti sulla propria tastiera) se si è specificata la codifica ISO-8859-1 in MikTeX:

Edit > Preferences > Editor > Encoding > ISO-8859-1

Altrimenti si possono usare le sequenze di escape (attenzione a quella per la lettera i accentata): à, è, é, ì, ò, ù.

Si ricordi che in Italiano l'accento è quasi sempre grave. L'accento acuto si usa in pochi casi specifici, quali le parole perché, poiché e finché, o la coppia né ... né.

I segni di interpunzione devono essere attaccati alla parola che li precede e separati con uno spazio dalla parola che li segue:

(giusto) “caspita, che bella notizia!”
(sbagliato) “caspita , che bella notizia!”
(sbagliato) “caspita ,che bella notizia!”

L'apostrofo deve essere attaccato sia alla parola che lo precede sia a quella che lo segue:

(giusto) “l'oro è un metallo prezioso”
(sbagliato) “l' oro è un metallo prezioso”
(sbagliato) “l 'oro è un metallo prezioso”
(sbagliato) “l ' oro è un metallo prezioso”

Lasciando una riga vuota si genera automaticamente un nuovo paragrafo, ossia si va a capo, si lascia un piccolo spazio verticale e si indenta la prima riga del paragrafo. E' importante cercare di organizzare il proprio testo in paragrafi che contengano insieme di frasi correlate. Quando si cambia argomento, si inizia un nuovo paragrafo. Si può considerare questo testo come un buon esempio di suddivisione in paragrafi.

L'uso del comando `\\` per forzare un ritorno a capo è fortemente deprecato. In sua vece bisogna usare il comando corrispondente all'effetto che si desidera ottenere: ad esempio, se è terminato un paragrafo basta lasciare una riga vuota, se si vuole creare un elenco basta usare uno dei comandi per la creazione di una lista.

Attenzione ai doppi apici che devono essere creati usando due volte il tipo di apice appropriato (aperto o chiuso), come nel seguente caso:

il film “Il Ciclone” è stato diretto ed interpretato da Leonardo Pieraccioni.

Se alcune parole ricorrono frequentemente nel testo e sono scritte in un modo particolare allora conviene definire un'opportuna abbreviazione nel preambolo del file, ossia prima di `\begin{document}`. Come esempi si vedano in questo file le definizioni e l'uso delle abbreviazioni `\ltx` per generare la parola `LATEX` e `\cmd` per presentare in modo opportuno i comandi.

E' anche facile creare delle note a piè di pagina¹ che vengono numerate automaticamente.

Non bisogna dimenticare di usare un correttore ortografico. Per installare il dizionario necessario scaricare dal sito <http://wiki.services.openoffice.org/wiki/Dictionaryes> il file ZIP corrispondente alla lingua prescelta (nel caso sia la lingua Inglese installare il dizionario British e non American) e quindi estrarre tutti i file nella cartella

`C:\Program Files\MiKTeX 2.9\hunspell\dicts`

Il correttore si attiva quindi scegliendo il menù:

Edit > Spelling > lingua_desiderata

¹...ma vanno usate con parsimonia.

Le parole errate compariranno evidenziate con una sottolineatura in rosso: sarà così possibile modificarle direttamente oppure – cliccandoci sopra col tasto destro – scegliere tra le opzioni di correzione proposte.

E' possibile scrivere parti della tesi in lingue diverse, specificando il linguaggio usato (importante in modo che il programma vada a capo in modo corretto). Segue una citazione in inglese:

The man in the rubber boots and a thick coat to protect against the evening chill walked purposefully about a farm here, scattering pheasants as he went. He could have been an English gentleman out for a bit of hunting, except he carried no gun.

In his current circumstance, the WikiLeaks founder Julian Assange is more hunted than hunter, fighting extradition to Sweden on accusations of sexual misconduct while struggling to maintain the influence of WikiLeaks even as he remains here at Ellingham Hall, the country manor house of Vaughan Smith, a former soldier and journalist who runs a restaurant and club for journalists in London.

Adesso si riprende il normale testo in Italiano, che seguirà le regole di composizione della lingua Italiana.

Per inserire citazioni testuali (ossia porzioni di testo) si può usare l'ambiente **quote** (citazione breve) oppure quello **quotation** (per citazioni lunghe, che posso essere composte da più paragrafi), come appena fatto qui sopra per la citazione in inglese.

Per inserire collegamenti a pagine o documenti web si usano i comandi `\url` e `\href` del package **hyperref** come nei due seguenti esempi:

uso di \href

(sorgente) Nel `\href{http://www.polito.it/}{sito web}` del Politecnico di Torino
(risultato) Nel sito web del Politecnico di Torino

uso di \url

(sorgente) Il sito web del Politecnico di Torino è `\url{http://www.polito.it/}`
(risultato) Il sito web del Politecnico di Torino è `http://www.polito.it/`

Infine è possibile citare testi o siti che sono stati consultati per la tesi: articoli a congresso [1], articoli su rivista [2], capitoli di libri [3], siti o pagine web [4] e RFC (come RFC-5246 [5] che definisce la versione 1.2 del protocollo TLS). Si noti che le voci della bibliografia devono essere elencate nell'apposita sezione (comando `\thebibliography`) secondo l'ordine in cui vengono citate. Per bibliografie più complesse è possibile l'uso di BibTex, ma è sconsigliato se non si supera la decina di citazioni.

1.3.2 Liste

Usare liste puntate per elenchi in cui l'ordine non è importante, come nel caso degli esami obbligatori da superare per essere ammessi al secondo anno:

- Analisi Matematica I;
- Fisica I;
- un esame a scelta tra Chimica ed Informatica.

Notare che normalmente ogni elemento della lista è costituito da un'unica frase, terminata da punto-e-virgola, tranne l'ultimo elemento che è terminato da un punto. Ogni punto inizia con lettera minuscola (a meno che la parola iniziale sia un sigla o un nome proprio). E' fortemente sconsigliato l'uso di una lista se uno o più punti contengono più di una singola frase. In questo caso conviene scrivere paragrafi separati oppure creare delle sottosezioni.

Le liste numerate si creano con l'ambiente **enumerate** e sono utili per elencare elementi aventi un ordine di priorità, come nel caso della ricetta per cucinare la pastasciutta:

1. prendere una pentola;
2. riempirla di acqua;
3. metterla sul fuoco e portare l'acqua ad ebollizione;
4. quando l'acqua bolle, buttare la pasta;
5. quando la pasta è cotta, scolarla, condirla e ... mangiarla!

Si possono anche fare delle liste che servono per descrivere generici oggetti, usando l'ambiente **description**. L'oggetto descritto viene automaticamente posizionato ad inizio riga e scritto in grassetto, come nel seguente esempio:

pentola oggetto metallico usato per cuocere cibi;

mela frutto che fa molto bene alla salute;

Sarchiapone animale immaginario usato in una gag di Walter Chiari e poi ripreso in un programma radiofonico da Renzo Arbore.

1.3.3 Formule matematiche ed unità di misura

Le formule matematiche si possono scrivere direttamente nel corpo del testo racchiudendole tra due simboli \$ (come in questo esempio per il calcolo della circonferenza, $c = 2\pi r$) oppure scrivere la formula su una riga centrata racchiudendola tra \[e \]:

$$c = 2\pi r$$

Apici, pedici, frazioni, puntini e sommatorie sono anche semplici da fare:

$$k = a_0 + a_1 \frac{1}{x} + a_2 \frac{1}{x^2} + \cdots = \sum_{i=0}^N a_i \frac{1}{x^i} = \sum_{i=0}^N a_i x^{-i}$$

Ecco ora un esempio di limite:

$$\lim_{x \rightarrow \infty} f(x) = 0$$

Quando si usano delle grandezze fare attenzione alle unità di misura: usare sempre il SI e scrivere correttamente i simboli (ad esempio, il simbolo del chilo è la lettera “k” minuscola non la “K” maiuscola). Per evitare errori e lasciare il giusto spazio tra quantità e simbolo si suggerisce l'uso del package **siunitx** che aggiunge i comandi `\si` per unità isolate o `\SI` per quantità numerica seguita dalla sua unità di misura. Ecco un esempio d'uso per citare unità isolate:

...fornire il risultato in cm².

ma è anche possibile indicare l'unità su quantità numeriche specifiche, citate in-line (ad esempio, un sacco di patate da 10 kg) o all'interno di una formula matematica:

$$v = \frac{s}{t} = \frac{100 \text{ m}}{5 \text{ s}} = 20 \text{ m/s}$$

Nel caso di unità relative al sistema binario è possibile adottare sia la vecchia notazione, che usa gli stessi prefissi del SI ma sottointende l'uso delle potenze di 2, sia la nuova notazione, in cui i prefissi del sistema binario sono caratterizzati da una “i” minuscola. Ad esempio:

$$1 \text{ TB (vecchia notazione)} = 1 \text{ TiB (nuova notazione)} = 2^{30} \text{ B}$$

1.3.4 Tabelle e figure

In $\text{\LaTeX}$ le tabelle si compongono descrivendo le righe di cui sono composte:

<i>modello</i>	<i>velocità</i> [km/h]	<i>consumi</i> [l/100 km]
Fiat 500	150	19
Alfa Giulietta	210	21.5
Ferrari Testarossa	320	5.7

Normalmente è meglio non inserire le tabelle direttamente nel testo ma creare un oggetto separato, con una didascalia ed un numero per citarlo ove necessario. Ad esempio è stata creata in questo modo la tabella 1.1 che riporta i voti conseguiti in alcuni esami da un ipotetico studente ma è slegata da questo specifico paragrafo. In questo modo $\text{\LaTeX}$ sistemerà la tabella in un punto opportuno del testo, evitando di lasciare spazio verticale inutilizzato. Si noti l'uso del carattere "tilde" (~) per tenere legato il numero della tabella (o figura o altro elemento flottante) alla parola che lo precede.

<i>esame</i>	<i>data</i>	<i>voto</i>
Analisi I	27/1/2009	18
Informatica	14/2/2009	30
Fisica I	15/7/2009	27

Table 1.1. Voti riportati negli esami sinora sostenuti.

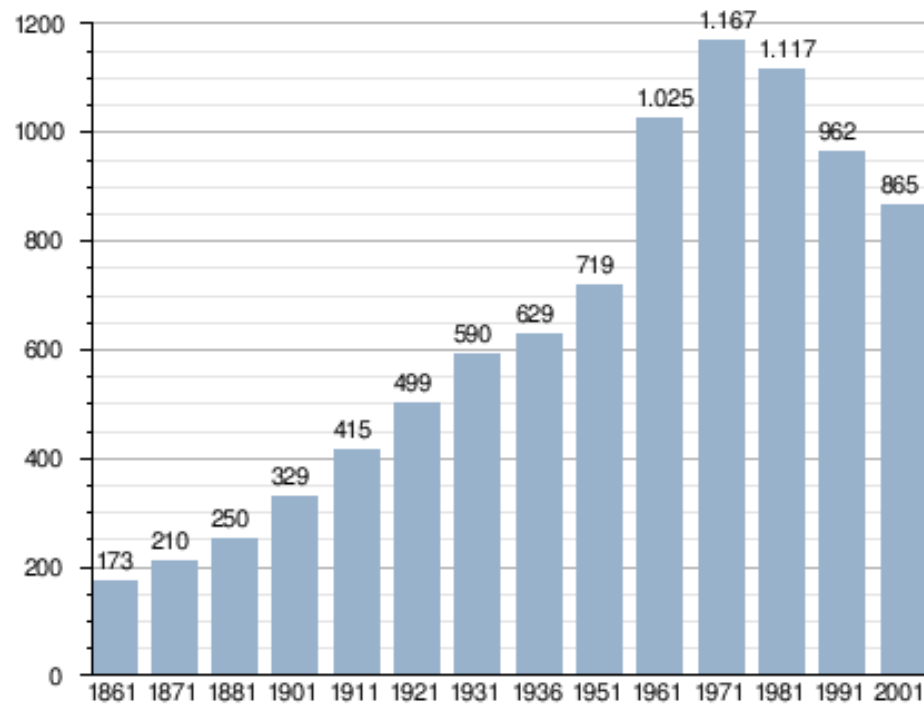
Per quanto riguarda le figure, conviene disegnarle con un apposito programma (es. Power-Point) e poi salvarle in un file che verrà richiamato nel file $\text{\LaTeX}$. Si consiglia di prestare attenzione ai font usati (suggeriti Arial o Helvetica), alla dimensione (minimo 10 pt) e di generare un formato grafico ad alta definizione (PDF è il preferito, altrimenti PNG o JPG ad alta qualità). Il file grafico verrà poi incluso in $\text{\LaTeX}$ per generare la figura, come nell'esempio della figura 1.1 che riporta il grafico della variazione di popolazione della città di Torino. Si noti che questo grafico è di bassa qualità (pixel percepibili ad occhio nudo) perché salvato come JPG a bassa risoluzione (solo 96 DPI). Per i formati raster si dovrebbe sempre usare una risoluzione di almeno 300 DPI. Nel caso che la figura non abbia già la dimensione desiderata si può ingrandirla o rimpicciolirla usando i parametri `width` (larghezza), `height` (altezza) o `scale`. Quando si indica larghezza o altezza conviene farlo come una frazione della pagina; ad esempio per avere una figura che occupa il 90% della larghezza pagina si scrive `width=0.9\textwidth` mentre si scrive `height=0.5\textheight` per una figura che deve occupare il 50% dell'altezza della pagina. Il parametro `scale` permette invece di indicare la dimensione come percentuale di quella originale della figura. Ad esempio se la figura originale contiene testo con font a 20 pt conviene scalarla al 50% (in modo che la figura abbia font a 10 pt) usando il parametro `scale=0.5`. L'uso di `scale` è fortemente consigliato nel caso che la figura contenga del testo, in modo da armonizzarne la dimensione con quello del corpo della pagina.

Se un'immagine è stata copiata da una fonte esterna, tale fonte deve essere indicata nella didascalia (come fatto ad esempio nella figura 1.1).

Nel caso che sia necessario includere del codice sorgente (cosa da fare con estrema parsimonia e solo in caso sia realmente necessario, ossia non includere codice solo per fare volume) si può usare l'ambiente `lstlisting` che include testo rispettandone la formattazione originale ed usando un font a spaziatura fissa, come nell'esempio in Fig. 1.2. E' anche possibile numerare le righe del programma, come nell'esempio in Fig. 1.3, ma la numerazione è da usarsi solo se nel testo si deve far riferimento a specifiche sezioni del programma.

Per citare piccoli pezzi di codice si può usare direttamente l'ambiente `lstlisting` nel testo (invece che in una figura separata), come nel seguente esempio relativo al codice HTML per centrare un testo:

```
<center>
```



fonte ISTAT - elaborazione grafica a cura di Wikipedia

Figure 1.1. Popolazione di Torino, in migliaia di abitanti (fonte: wikipedia).

```
#include <stdio.h>

int main ()
{
    printf ("Ciao!\n");
    return 0;
}
```

Figure 1.2. Esempio di programma inserito tramite `lstlisting`.

```
Esempio di testo centrato.
</center>
```

Quando invece si vuol citare del codice (molto corto) all'interno di una riga si può usare `\code` come in questo caso in cui dico che in HTML per centrare del testo si può usare il tag `<center>` ma è deprecato (meglio usare uno stile CSS).

1.4 La bibliografia

Anche se siamo nell'era di Internet e dei motori di ricerca, è buona norma citare con precisione le fonti a cui si è attinto per scrivere la tesi.

I nomi degli autori devono essere specificati con l'iniziale del nome, seguita da punto e quindi dal cognome.

```
1 #include <stdio.h>
2
3 int main ()
4 {
5     printf ("Ciao!\n");
6     return 0;
7 }
```

Figure 1.3. Esempio di programma inserito tramite `\lstlisting` con numerazione delle righe.

Nel caso di articoli su rivista, deve essere indicato il titolo dell'articolo, il nome della rivista, il numero del fascicolo (solo se la rivista è numerata), il mese e l'anno di pubblicazione, la pagina di inizio e fine dell'articolo.

Nel caso di articoli pubblicati a congresso, convegno o workshop, deve essere indicato il titolo dell'articolo, il nome del congresso, il luogo (città e nazione), la data e la pagina di inizio e fine dell'articolo.

Nel caso di libri si indicano gli autori, il titolo del libro, l'editore e l'anno di pubblicazione. Se si vuole citare uno specifico capitolo o insieme di pagina, si inserisce tale indicazione nel testo del documento e non nella voce bibliografica.

Se si citano documenti pubblici (es. standard, RFC, report tecnici, pagine web) occorre fornire oltre ai dati identificativi anche la URL tramite cui è possibile accedere al documento.

Qualunque sia la tipologia di articolo, è molto importante citare – se disponibile – il *DOI* (Digital Object Identifier). Questo è un codice universale che identifica univocamente una pubblicazione, sia essa a stampa o in formato elettronico. Specificando il DOI come se fosse una pagina all'interno del sito <http://dx.doi.org/> si viene ridiretti automaticamente alla pubblicazione corrispondente. Normalmente tutte le pubblicazioni recenti e di qualità hanno un DOI assegnato, mentre per quelle più vecchie o di minore qualità il DOI è raramente disponibile.

Si veda la bibliografia presente in questo testo come esempio di corretta e completa citazione di vari tipi di riferimenti bibliografici. Si noti che ogni elemento della bibliografia non è terminato dal carattere punto.

Le citazioni si inseriscono nel testo usando il comando `\cite` seguito in parentesi graffe dalla sigla usata per identificare il riferimento bibliografico. Il comando `\cite` deve essere separato con uno spazio dalla parola che lo precede:

```
(giusto) il protocollo TLS \cite{tls12} è usato per la sicurezza del web
(sbagliato) il protocollo TLS\cite{tls12} è usato per la sicurezza del web
```

In questo esempio `tls12` è la sigla usata nella sezione della bibliografia come identificativo mnemonico dello standard TLS (per il dettaglio del formato bibliografico si veda il relativo sorgente nel file `biblio.tex`).

1.5 Acronimi

E' possibile usare il package "glossaries" per riportare automaticamente gli acronimi nell'apposito indice. Per aggiungere nuovi acronimi è sufficiente aprire nel file `acronyms.tex` e usare il seguente comando:

```
\newacronym{acronym}{ACRONYM}{Definition}
```

Esempio di utilizzo:

`\newacronym{vr}{VR}{Virtual Reality}`

Essi possono essere richiamati nel testo nel seguente modo:

- Comando `\glstrfull{vr}`: richiama sia la definizione dell'acronimo che l'acronimo stesso (es. Virtual Reality (VR))
- Comando `\glstrlong{vr}`: richiama la sola definizione dell'acronimo (es. Virtual Reality)
- Comando `\glstrshort{vr}`: richiama il solo acronimo (es. VR)

Aggiungendo il suffisso "pl" ai suddetti comandi è possibile ottenere la versione plurale della definizione e dell'acronimo:

- Esempio di output del comando `\glstrfullpl{vr}`: Virtual Realities (VRs)
- Esempio di output del comando `\glstrlongpl{vr}`: Virtual Realities
- Esempio di output del comando `\glstrshortpl{vr}`: VRs

Com'è possibile notare, per alcuni casi il plurale di una parola, ottenuto aggiungendo la sola s al termine, potrebbe non essere corretto. In tal caso, occorre dichiarare il plurale corretto nel seguente modo:

`\newacronym[shortplural={VRs},longplural={Virtual Realities}]{vr}{VR}{Virtual Reality}`

Seguono esempi di plurali di definizioni di acronimi e acronimi corretti:

- Esempio di output del comando `\glstrfullpl{ghg}`: Greenhouse Gases (GHGs)
- Esempio di output del comando `\glstrlongpl{ghg}`: Greenhouse Gases
- Esempio di output del comando `\glstrshortpl{ghg}`: GHGs

1.6 Conclusioni

Queste sono solo delle brevi note sull'uso di $\LaTeX$ per comporre semplici testi. Per necessità più complesse si consiglia di consultare leggere la documentazione che viene installata assieme a $\LaTeX$. In particolare con MikTeX (versione 2.9) la documentazione viene installata nella cartella

`C:\Program Files\MikTeX 2.9\doc`

ed in particolare quella relativa ai vari package nella sottocartella `latex\nome-del-package`; ad esempio, la documentazione del package `siunitx` si trova nella cartella

`C:\Program Files\MikTeX 2.9\doc\latex\siunitx`

Inoltre è sempre possibile consultare l'ampia documentazione disponibile in rete.

Chapter 2

La tesi di laurea magistrale

2.1 Struttura della tesi

Normalmente la tesi per la laurea magistrale è composta da almeno cinque capitoli.

Il primo capitolo contiene l'introduzione e descrive il contesto o il problema da cui è scaturito l'argomento della tesi. In altre parole, fornisce una giustificazione per lo svolgimento della tesi e dovrebbe incuriosire il lettore a scoprire come il problema è stato affrontato e risolto.

Nel caso di una tesi svolta in azienda è possibile inserire anche una breve descrizione dell'azienda o ente presso cui è stata svolta, con particolare riferimento al contesto applicativo, ingegneristico o commerciale da cui è nata la necessità alla base della tesi.

Il secondo capitolo fornisce – se necessario – un'analisi critica dei lavori precedenti sul tema trattato nella tesi.

Il terzo capitolo descrive l'approccio usato per risolvere il problema. Ad esempio, nel caso di progettazione di un'applicazione web, si possono fornire le specifiche delle singole pagine, il collegamento tra le varie pagine, la struttura del database e la logica applicativa. Questo capitolo può anche essere spezzato in più capitoli se dovesse risultare particolarmente voluminoso, dedicando ciascun capitolo ad uno specifico tema (es. la progettazione del database, la logica applicativa).

Il capitolo successivo fornisce i risultati dell'attività svolta, cercando di misurare i risultati in modo quantitativo.

Infine il capitolo conclusivo riassume i risultati conseguiti ed indica se ci sono ulteriori possibili sviluppi o miglioramenti.

2.2 Writing the thesis in English

If the thesis is written in English, then some terms should be changed (such Capitolo that should become Chapter) and hyphenation should follow the rules of the English language.

To achieve this effect, please insert `\selectlanguage{english}` in the preamble (that is, before `\begin{document}`) and do not forget to select the UK-English dictionary for the spell checker.

Chapter 3

Progettazione

Discutere in questo capitolo come è stata progettata la soluzione al problema trattato nella tesi, indicando anche se sono stati valutati vari possibili approcci o soluzioni pre-esistenti e giustificando le proprie scelte. Descrivere quindi la soluzione vera e propria.

Nel caso sia stato sviluppato del software non triviale, è buona norma dedicargli tre sezioni:

- architettura dell'applicazione (interazioni con gli utenti e con altri sistemi, moduli logici, flussi dati interni ed esterni);
- manuale dello sviluppatore (descrizione dei moduli, degli algoritmi, delle interfacce e delle strutture dati);
- manuale utente (come installare ed usare il programma, interfacce, comandi, dati in input ed in output).

Nel caso di software molto voluminoso, queste tre sezioni possono diventare tre capitoli separati.

Chapter 4

Risultati

Inserire in questo capitolo i risultati conseguiti, cercando di analizzarli – se possibile – in modo quantitativo.

Chapter 5

Conclusioni

Qui si inseriscono brevi conclusioni sul lavoro svolto, senza ripetere inutilmente il sommario.

Si possono evidenziare i punti di forza e quelli di debolezza, nonché i possibili sviluppi futuri o attività da svolgere per migliorare i risultati.

Chapter 6

Foundations

The scope of our software is to provide a highly configurable and interactive simulation environment specifically designed for the analysis of Comet dust tails. Cometary dust tails are complex astrophysical phenomena formed when solar heating sublimates the volatile ices within a comet’s nucleus, dragging dust particles into space. As such, the software must be capable of accurately simulating the dynamics of myriad individual dust particles within the tail, taking into account a wide array of physical forces. These forces prominently include the gravitational pull of the Sun and planets, as well as the solar radiation pressure, which is often the dominant force dictating the sweeping shape of the tail. Furthermore, the simulation must inherently couple these particle dynamics with the precise orbital motion and potential rotation of the Comet nucleus itself. This chapter is therefore centered around the mathematical and physical foundations that underpin the software. It details the equations of motion governing the dust particles, comprehensively defines the forces acting upon them, and critically evaluates the numerical methods employed to solve these complex differential equations. We will also explicitly discuss the fundamental assumptions, boundary conditions, and physical simplifications made in our mathematical model, addressing the inherent limitations of our approach when compared to real-world observational data.

6.1 Numerical Methods for Differential Equations

6.1.1 Ordinary Differential Equations

An Ordinary Differential Equation (ODE) is a mathematical equation that establishes a relationship between an unknown continuous function of one independent variable—typically representing time, t , in physical simulations—and its derivatives. A first-order ODE, or a generalized system of such equations, can be elegantly expressed in vector notation as:

$$\dot{\mathbf{y}}(t) = \mathbf{f}(t, \mathbf{y}(t)) \tag{6.1}$$

where $\mathbf{y}(t) \in \mathbb{R}^n$ defines the state vector encompassing all dynamic variables of the system, and $\mathbf{f} : \mathbb{R} \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ represents the vector field describing the instantaneous rate of change of the state based on the current time and state configuration. For an Initial Value Problem (IVP), where the state is known at an initial time t_0 as $\mathbf{y}(t_0) = \mathbf{y}_0$, to be uniquely solvable, the defining function $\mathbf{f}$ must satisfy a condition known as Lipschitz continuity with respect to the state variable $\mathbf{y}$. Lipschitz continuity dictates that there exists a real constant $L \geq 0$ such that $\|\mathbf{f}(t, \mathbf{y}_1) - \mathbf{f}(t, \mathbf{y}_2)\| \leq L\|\mathbf{y}_1 - \mathbf{y}_2\|$ for all $\mathbf{y}_1, \mathbf{y}_2$ in the domain. This is a significantly stronger mathematical condition than standard uniform continuity, ensuring that the rate of change of the vector field $\mathbf{f}$ is strictly bounded. This property is absolutely fundamental to simulation stability because it guarantees, via the Picard-Lindelöf theorem, that solution trajectories originating from distinct initial states will not intersect, nor will they diverge to infinity in a finite amount of time, thereby ensuring the existence and uniqueness of the physical trajectory [6].

6.1.2 Vector Spaces and Explicit Methods

When the physical state variable $\mathbf{y}$ resides exclusively within a standard vector space (e.g., Euclidean space $\mathbb{R}^n$), the fundamental algebraic operations of vector addition and scalar multiplication are globally well-defined and straightforward. This consistent algebraic structure is what enables the deployment of classical numerical integration schemes. Explicit methods iteratively compute the discrete state at the next forward time step $t_{k+1} = t_k + h$ (where h denotes the discrete integration time step size) using strictly evaluated information from the current or previously computed steps, avoiding the need to solve algebraic equations.

- **Explicit Euler:** The simplest and most intuitive explicit method, conceptually derived from a first-order Taylor series expansion. It updates the state linearly as $\mathbf{y}_{k+1} = \mathbf{y}_k + h\mathbf{f}(t_k, \mathbf{y}_k)$. As a first-order method, its local truncation error is $\mathcal{O}(h^2)$ and its global error is $\mathcal{O}(h)$. Consequently, it often suffers from extremely poor accuracy and severe numerical instability, particularly when applied to stiff differential equations where small perturbations amplify rapidly.
- **Runge-Kutta Methods:** A broad family of higher-order iterative methods that significantly improve integration accuracy by strategically evaluating the vector field at multiple intermediate sample points within the interval $[t_k, t_{k+1}]$. The classic fourth-order Runge-Kutta (RK4) method is the workhorse of numerical integration, achieving a global accuracy of $\mathcal{O}(h^4)$. It does so by calculating a weighted average of four estimated slopes: the initial slope, two intermediate midpoint slopes, and a final slope at the end of the interval.

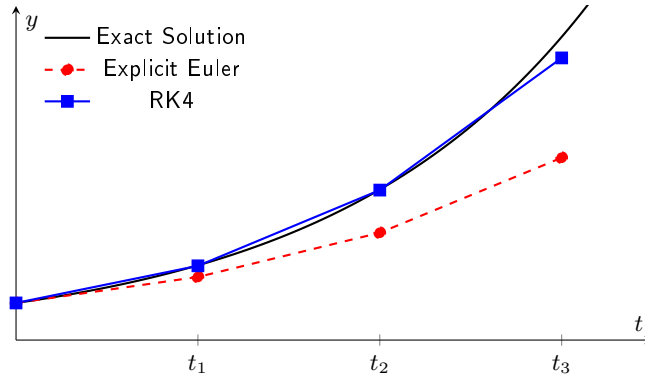


Figure 6.1. Comparison of numerical integration methods. Explicit Euler unconditionally diverges for exponentially growing or stiff systems, while higher-order methods gracefully track the true continuous solution over short intervals.

It is crucial to emphasize that the mathematical validity of these classical explicit methods relies intrinsically on the global vector space structure. They mathematically construct the subsequent state by linearly scaling and directly adding derivative vectors ($h\mathbf{f}$) to the current base state ($\mathbf{y}_k$). As will be detailed later, if the simulated state manifold does not constitute a valid vector space (for example, if the state is constrained to represent a valid 3D rotation), naively adding Euclidean components will immediately force the state off the valid geometric manifold, leading to corrupted, non-physical states.

6.1.3 Systems of ODEs and Higher-Order ODEs

In the realm of classical physics, dynamic systems are predominantly described by higher-order differential equations. The most ubiquitous example is Newton's second law of motion, $\ddot{\mathbf{x}} = \mathbf{F}(t, \mathbf{x}, \dot{\mathbf{x}})/m$, which is inherently a second-order ODE. Fortunately, these higher-order constructs can be seamlessly handled using the exact same first-order integration techniques by systematically reducing them to a higher-dimensional system of first-order ODEs. By defining a composite phase-space state vector that explicitly includes the lower-order time derivatives, such as $\mathbf{y} =$

$[\mathbf{x}^T, \dot{\mathbf{x}}^T]^T = [\mathbf{x}^T, \mathbf{v}^T]^T$, the generic second-order equation is elegantly transformed and rewritten as a coupled first-order system:

$$\dot{\mathbf{y}} = \begin{bmatrix} \dot{\mathbf{x}} \\ \dot{\mathbf{x}} \end{bmatrix} = \begin{bmatrix} \mathbf{v} \\ \frac{1}{m} \mathbf{F}(t, \mathbf{x}, \mathbf{v}) \end{bmatrix} \quad (6.2)$$

This expanded state space representation is universally applicable, allowing any generic explicit or implicit ODE solver to be applied entirely transparently to physical systems of arbitrary dimension, complexity, and differential order.

6.1.4 Symplectic Integrators and Error Bounds

While standard explicit methods like RK4 offer highly accurate short-term local trajectories, they fundamentally fail to preserve the underlying geometric structure and symmetries of conservative Hamiltonian systems. In protracted physical simulations, such as orbital mechanics over thousands of years, this geometric failure manifests as a gradual, non-physical drift in the total mechanical energy—often leading to artificial orbital decay or expansion.

Symplectic integrators represent a specialized class of numerical methods designed specifically to natively preserve the symplectic two-form of the Hamiltonian phase space (satisfying Liouville's theorem of phase space volume conservation). To mathematically understand their stability, we rely on *Backward Error Analysis* (BEA). BEA reveals that instead of calculating an approximate, drifting solution to the exact continuous Hamiltonian H , a symplectic integrator computes the *exact* continuous solution to a closely related "shadow" or modified Hamiltonian defined as $\tilde{H} = H + \mathcal{O}(h^p)$, where p is the order of the method.

In practical terms, this geometric preservation translates directly into the rigorous bounding of global energy errors. Instead of the total energy drifting monotonically over time (as occurs with standard explicit methods), the numerically computed energy of a symplectic integrator oscillates tightly around the true, exact energy level. This oscillation is strictly bounded by an error of $\mathcal{O}(h^p)$ for exponentially long times ($\mathcal{O}(e^{c/h})$), ensuring unparalleled long-term stability and physical fidelity. Two notable, highly efficient symplectic methods include:

- **Leapfrog Integration:** A second-order accurate method that evaluates positional and velocity data at interleaved, staggered time steps. It sequentially updates velocities at half-steps ($t_{k+1/2}$) and positions at integer steps (t_{k+1}). This alternating update scheme intrinsically provides excellent long-term energy stability while remaining computationally inexpensive.
- **Velocity Verlet:** A mathematically equivalent variation of the Leapfrog method that evaluates both positions and velocities at synchronized integer time steps. This formulation minimizes memory overhead and simplifies the integration with position-dependent forces, making it the de facto standard in molecular dynamics and complex rigid body physics engines.

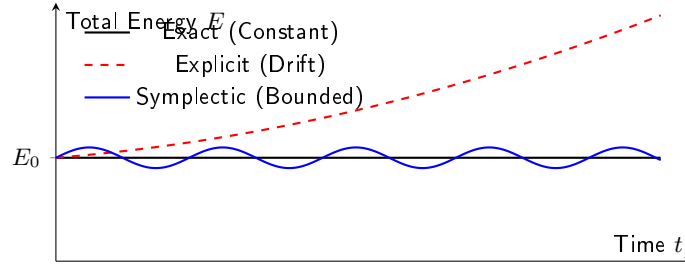


Figure 6.2. Energy conservation in conservative systems. Symplectic methods exhibit strictly bounded oscillations tightly around the exact initial energy, elegantly avoiding the secular, non-physical drift characteristic of standard explicit methods over long durations.

6.2 Rigid Body Dynamics

Having established a robust framework of numerical techniques required to simulate generalized differential equations, we now turn our focus to the specific physical laws governing rigid body dynamics. A rigid body is a theoretical idealization of a solid macroscopic object in which the spatial distance between any two internal constituent points remains perfectly constant, regardless of the magnitude or direction of external forces and torques applied to it.

6.2.1 Translational Dynamics

The translational motion of a rigid body, ignoring its orientation, is mathematically identical to that of an idealized point mass concentrated entirely at the body's geometric center of mass.

- **Linear Momentum:** Formally defined as $\mathbf{p} = m\mathbf{v}$, where m represents the total scalar mass of the rigid body and $\mathbf{v}$ denotes the instantaneous velocity vector of its center of mass relative to an inertial reference frame.
- **Newton's Second Law:** The fundamental principle stating that the time derivative of linear momentum is directly proportional to, and strictly equals, the vector sum of all external forces applied to the body: $\dot{\mathbf{p}} = \mathbf{F}_{\text{ext}}$. In the absence of net external forces, linear momentum is strictly conserved.
- **Energy:** The total mechanical energy E of a conservative system is the sum of its kinetic and potential energies. The translational component of the kinetic energy is directly proportional to the square of the velocity magnitude: $K_{\text{trans}} = \frac{1}{2}m\|\mathbf{v}\|^2$.

6.2.2 Rotational Dynamics

The rotational behavior of a rigid body introduces complex angular counterparts to the translational equations, heavily dependent on the object's mass distribution:

- **Angular Velocity ($\boldsymbol{\omega}$):** A 3D pseudovector (or axial vector) whose spatial direction dictates the instantaneous axis of rotation, and whose magnitude determines the rate of angular rotation in radians per second.
- **Center of Mass:** The unique internal equilibrium point where the weighted relative position vectors of the body's distributed mass sum precisely to zero. All unconstrained rotations naturally occur around this point.
- **Inertia Tensor ($\mathbf{I}$):** The intricate rotational equivalent of scalar mass. It is formulated as a 3×3 symmetric, positive-definite matrix that encapsulates exactly how the rigid body's mass is spatially distributed relative to the center of mass. The principal axes of this tensor define the natural axes of stable and unstable rotation.
- **Angular Momentum ($\mathbf{L}$):** The rotational analog to linear momentum, defined mathematically as the matrix-vector product $\mathbf{L} = \mathbf{I}\boldsymbol{\omega}$. The universally fundamental principle of conservation of angular momentum dictates that $\mathbf{L}$ remains absolutely constant over time if no net external torque acts upon the system.
- **Torque ($\boldsymbol{\tau}$):** The rotational equivalent of a linear force, representing a twisting force that causes a change in angular momentum over time, governed by Euler's rotational equation: $\dot{\mathbf{L}} = \boldsymbol{\tau}_{\text{ext}}$. Expanded in the body frame, this yields the non-linear Euler equations $\mathbf{I}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega}) = \boldsymbol{\tau}_{\text{ext}}$.
- **Total Kinetic Energy:** The complete kinetic energy of an unconstrained rigid body is the scalar sum of its independent translational and rotational kinetic energies: $K_{\text{total}} = \frac{1}{2}m\|\mathbf{v}\|^2 + \frac{1}{2}\boldsymbol{\omega}^T \mathbf{I} \boldsymbol{\omega}$.

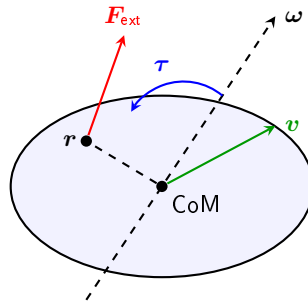


Figure 6.3. Kinematics and dynamics of a rigid body. An external force F_{ext} applied at relative position r induces both a linear acceleration of the Center of Mass (CoM) and a torque τ modifying the angular velocity ω .

6.2.3 Inertia Tensor Computation

The inertia tensor $\mathbf{I}$ of a continuous rigid body is a symmetric 3×3 matrix that quantitatively describes its mass distribution and determines the torque required for a desired angular acceleration about a given rotational axis. For a continuous body with mass density $\rho(\mathbf{r})$ distributed over a volume V , the inertia tensor evaluated at the origin is rigorously defined by the volume integral:

$$\mathbf{I} = \iiint_V \rho(\mathbf{r}) (\|\mathbf{r}\|^2 \mathbf{1}_{3 \times 3} - \mathbf{r} \mathbf{r}^T) dV \quad (6.3)$$

where $\mathbf{r} = [x, y, z]^T$ is the position vector of the infinitesimal mass element $dm = \rho(\mathbf{r})dV$ relative to the origin, and $\mathbf{1}_{3 \times 3}$ is the identity matrix. Expanding the matrix components yields the diagonal *moments of inertia* (e.g., $I_{xx} = \iiint \rho(y^2 + z^2)dV$) and the off-diagonal *products of inertia* (e.g., $I_{xy} = -\iiint \rho xy dV$).

In practical physics engines, bodies are often composed of primitive shapes (spheres, boxes, cylinders) or convex polyhedra. For these shapes, analytical solutions or discrete surface integrals exist to compute the local inertia tensor exactly around their respective centers of mass. When aggregating multiple shapes into a single composite rigid body, or when the rotation axis is shifted away from the center of mass, we employ the **Parallel Axis Theorem** (Steiner's theorem). For a body of mass m and a center of mass inertia tensor $\mathbf{I}_c$, the translated inertia tensor $\mathbf{I}'$ relative to a new parallel coordinate frame displaced by a vector $\mathbf{d}$ is given by:

$$\mathbf{I}' = \mathbf{I}_c + m (\|\mathbf{d}\|^2 \mathbf{1}_{3 \times 3} - \mathbf{d} \mathbf{d}^T) \quad (6.4)$$

This algebraic transformation avoids computationally expensive numerical integration by shifting previously known, exact primitive tensors to a common shared coordinate frame.

6.3 Rotational Representations and Lie Groups

6.3.1 Introduction to Lie Groups

To numerically integrate complex rotational dynamics accurately over time, we must first establish a mathematically robust representation for 3D spatial rotations. From an intuitive engineering and physics perspective, a Lie group acts as a continuous, smooth mathematical manifold—a curved topological space that, on a purely local level, resembles flat Euclidean space—but simultaneously possesses the rigid algebraic structure of a group, meaning elements can be continuously multiplied and smoothly inverted. When simulating unconstrained 3D rotations, we predominantly focus on two specific, interrelated Lie groups:

- **Special Orthogonal Group $\text{SO}(3)$:** The classical matrix group encompassing all 3×3 orthogonal matrices that strictly satisfy the condition $\mathbf{R}^T \mathbf{R} = \mathbf{I}$ with a positive determinant of $\det(\mathbf{R}) = +1$. This directly represents length-preserving and handedness-preserving rigid rotations.
- **Unit Quaternions $\text{Sp}(1)$ or $\text{SU}(2)$:** A specific mathematical subset of 4D hypercomplex numbers forming a closed unit hypersphere (denoted S^3) in 4D space. Represented as $\mathbf{q} = [q_w, q_x, q_y, q_z]$, unit quaternions offer an extremely compact, computationally efficient, and intrinsically singularity-free representation. Crucially, they completely avoid the "Gimbal lock" phenomenon that plagues traditional Euler angle representations.

6.3.2 Lie Algebras and the Hat Operator

The tangent vector space evaluated precisely at the identity element of any Lie group forms its corresponding Lie algebra. This algebraic structure elegantly represents the space of all possible infinitesimal rotational transformations (i.e., rotational velocities). For the specific case of $\text{SO}(3)$, its Lie algebra, denoted as $\mathfrak{so}(3)$, corresponds to the mathematical space of all 3×3 skew-symmetric

matrices. The instantaneous angular velocity pseudovector $\boldsymbol{\omega}$ inherently and exclusively resides within this specific algebra. To rigorously bridge the gap between 3D vectors and matrices, we define the "hat" operator $\widehat{(\cdot)}$ to uniquely map a standard 3D pseudovector $\boldsymbol{\omega} \in \mathbb{R}^3$ directly to its exact matrix representation within $\mathfrak{so}(3)$:

$$\widehat{\boldsymbol{\omega}} = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix} \quad (6.5)$$

When mathematically applying this hat-transformed matrix to another generic spatial vector, it perfectly retro-projects the operation, acting identically to a standard vector cross product: $\widehat{\boldsymbol{\omega}}\mathbf{v} = \boldsymbol{\omega} \times \mathbf{v}$. For unit quaternions, the derivative mapping from the physical angular velocity $\boldsymbol{\omega}$ to the mathematical rate of change of the quaternion itself, $\dot{\mathbf{q}}$, requires a specialized matrix hat operator acting dynamically on the 4D state space. This mapping typically takes the compact algebraic form $\dot{\mathbf{q}} = \frac{1}{2}\widehat{\boldsymbol{\omega}}_q\mathbf{q}$, where the operator $\widehat{\boldsymbol{\omega}}_q$ is systematically constructed using the 3D spatial components of $\boldsymbol{\omega}$.

6.3.3 Numerical Integration on Manifolds

Given these non-Euclidean geometries, it becomes fundamentally apparent that numerical integration algorithms designed purely for flat vector spaces require the intermediary use of Lie algebras to function correctly. If one were to naively attempt to use Explicit Euler to directly update a rotation matrix by adding a scaled linear derivative ($\mathbf{R}_{k+1} = \mathbf{R}_k + h\dot{\mathbf{R}}$), the resulting updated matrix $\mathbf{R}_{k+1}$ would definitively no longer be strictly orthogonal. By simply adding matrices, the state violently leaves the highly constrained $\text{SO}(3)$ manifold.

To circumvent this, we must formally map the current rotational state into the locally flat Lie algebra using the hat operator, perform the linear numerical integration safely within the valid, unconstrained vector space of the algebra, and ultimately project the resultant state back onto the curved Lie group manifold. This final projection is flawlessly executed utilizing the exact matrix exponential map, denoted $\exp : \mathfrak{so}(3) \rightarrow \text{SO}(3)$. For a rotation vector $\Delta\boldsymbol{\theta} = \boldsymbol{\omega}h$ with magnitude $\theta = \|\Delta\boldsymbol{\theta}\|$ and normalized axis $\mathbf{u} = \Delta\boldsymbol{\theta}/\theta$, the closed-form evaluation of this matrix exponential yields the renowned Rodrigues' rotation formula:

$$\exp(\widehat{\Delta\boldsymbol{\theta}}) = \mathbf{1}_{3 \times 3} + \sin(\theta)\widehat{\mathbf{u}} + (1 - \cos(\theta))\widehat{\mathbf{u}}^2 \quad (6.6)$$

The discrete update for the rotation matrix then becomes a multiplicative Lie group operation: $\mathbf{R}_{k+1} = \mathbf{R}_k \exp(\widehat{\boldsymbol{\omega}}h)$.

An analogous, highly efficient exponential map exists for unit quaternions mapping from $\mathfrak{su}(2) \rightarrow \text{SU}(2)$:

$$\exp_q(\Delta\boldsymbol{\theta}) = \left[\cos\left(\frac{\theta}{2}\right), \mathbf{u} \sin\left(\frac{\theta}{2}\right) \right]^T \quad (6.7)$$

where the update is performed using quaternion multiplication: $\mathbf{q}_{k+1} = \mathbf{q}_k \otimes \exp_q(\boldsymbol{\omega}h)$. This comprehensive methodology, broadly known as Lie Group Integration, geometrically guarantees the state remains exactly on the valid mathematical manifold at all times without requiring arbitrary, physically unmotivated artificial re-normalization steps.

6.4 Implicit Integration Methods

6.4.1 Root-Finding and Newton-Raphson

While explicit integration methods are computationally straightforward and exceptionally easy to implement, they suffer from a fatal flaw when applied to "stiff" differential equations—systems where physical components naturally evolve on vastly disparate, competing timescales. For stiff systems, explicit methods require prohibitively infinitesimal time steps to avoid catastrophic numerical explosion. Implicit methods gracefully resolve this instability by radically changing the

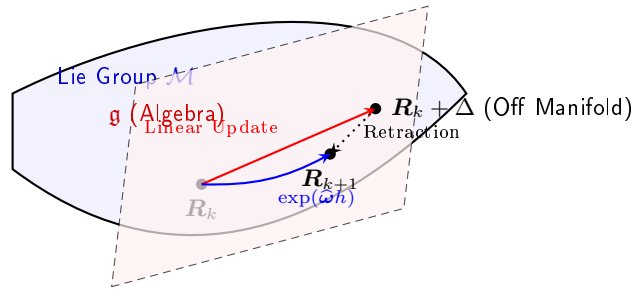


Figure 6.4. Lie Group Integration. Standard linear updates push the state off the curved manifold. Lie group methods utilize the local Lie algebra and the exponential map to seamlessly advance the state along the true geometric surface.

evaluation point: they evaluate the requisite vector field at the yet-unknown future time step t_{k+1} . Mathematically, this necessitates formulating and solving a complex nonlinear algebraic equation of the form $\mathbf{F}(\mathbf{y}_{k+1}) = \mathbf{0}$ at every single integration step. The Newton-Raphson algorithm is a dominant, universally applied iterative root-finding technique tailored for solving such nonlinear systems. It utilizes local derivative information to iteratively converge on the true solution:

$$\mathbf{y}_{k+1}^{(i+1)} = \mathbf{y}_{k+1}^{(i)} - \left[\nabla \mathbf{F}(\mathbf{y}_{k+1}^{(i)}) \right]^{-1} \mathbf{F}(\mathbf{y}_{k+1}^{(i)}) \quad (6.8)$$

where $\nabla \mathbf{F}$ denotes the exact, analytically derived Jacobian matrix of the system.

For complex physical simulations where the analytical Jacobian is either mathematically intractable to derive or overwhelmingly computationally expensive to continually evaluate and invert, quasi-Newton secant approximations—such as the renowned Broyden method—can be seamlessly substituted. Broyden’s method iteratively and efficiently approximates the Jacobian $\mathbf{B} \approx \nabla \mathbf{F}$ (or its direct inverse) on-the-fly using only the function evaluations from consecutive steps. Defining the state difference $\Delta \mathbf{y} = \mathbf{y}^{(i+1)} - \mathbf{y}^{(i)}$ and the function difference $\Delta \mathbf{F} = \mathbf{F}(\mathbf{y}^{(i+1)}) - \mathbf{F}(\mathbf{y}^{(i)})$, the secant update for the approximate Jacobian is:

$$\mathbf{B}_{i+1} = \mathbf{B}_i + \frac{\Delta \mathbf{F} - \mathbf{B}_i \Delta \mathbf{y}}{\|\Delta \mathbf{y}\|^2} \Delta \mathbf{y}^T \quad (6.9)$$

This approach sacrifices exact quadratic convergence for superlinear convergence, dramatically drastically reducing the per-iteration computational cost by eliminating the need to re-evaluate the full analytical Jacobian matrix.

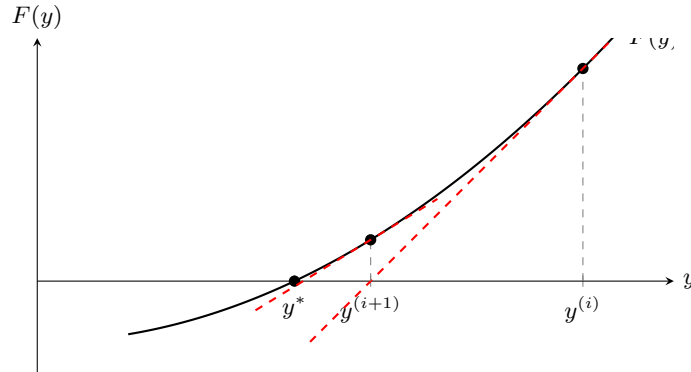


Figure 6.5. The Newton-Raphson method for root-finding. By iteratively evaluating the function and its local derivative (tangent line, dashed red), the sequence of estimates $y^{(i)}$ rapidly converges towards the true root y^* .

6.4.2 Implicit Integration

Equipped with powerful numerical root-finding capabilities, we can successfully deploy advanced, highly robust implicit integrators. The Implicit Midpoint Rule serves as an exemplary gold standard for such methods:

$$\mathbf{y}_{k+1} = \mathbf{y}_k + h \mathbf{f} \left(t_k + \frac{h}{2}, \frac{\mathbf{y}_k + \mathbf{y}_{k+1}}{2} \right) \quad (6.10)$$

Rather than evaluating at the boundaries, this elegant formulation evaluates the state derivative precisely at the temporal midpoint of the integration step, utilizing the arithmetic average of the known current state and the unknown future state. Mathematically, the Implicit Midpoint Rule is unconditionally A-stable for linear systems (meaning integration will not mathematically explode regardless of step size), is formally second-order accurate in time, and, perhaps most critically for physics engines, belongs to the elite class of symplectic integrators. This rare trifecta—combining unconditional stability with rigorous geometric energy preservation—makes it an exceptionally favorable and deeply reliable choice for stiff, long-duration physical simulations where both unwavering numerical stability and strict adherence to conservation laws are paramount.

6.5 Coupled Particle and Rigid Body Integration

To construct a robust, mathematically rigorous integration scheme for a mutually coupled system of particles and a rigid body, we must unify two powerful geometric integrators: Velocity Verlet (VV) for the Euclidean particle space $\mathbb{R}^{3N}$ and the Implicit Midpoint Rule (IMR) on a Lie Group for the rigid body manifold $\mathbb{R}^3 \times \text{SO}(3)$. Both integrators share two fundamental properties: they are symmetric (time-reversible) and second-order accurate $\mathcal{O}(h^2)$. However, Velocity Verlet is explicit in its position updates, whereas IMR is fully implicit. By interleaving the explicit kicks of the particles with the implicit Lie group solve of the rigid body, we create a Partitioned Implicit-Explicit (IMEX) scheme. This synchronizes all force evaluations exactly at the time midpoint $t_{n+1/2}$, guaranteeing symplectic-like structure preservation while restricting the expensive implicit root-finding purely to the 6×6 rigid body space. Here is the in-depth mathematical derivation of the continuous formulation, the coupled discrete algorithm, the analytical $\text{SO}(3)$ Jacobian, and the proof of reversibility.

6.5.1 Configuration Manifolds and Continuous Dynamics

Let the system consist of N particles and a single rigid body. We decouple the rigid body's Euclidean translation from its non-Euclidean rotation to avoid the translation-rotation velocity coupling inherent to the $\text{SE}(3)$ exponential map.

Particle System ($\mathbb{R}^{3N}$):

For particles with positions $\mathbf{q} \in \mathbb{R}^{3N}$, velocities $\mathbf{v} \in \mathbb{R}^{3N}$, and diagonal mass matrix $\mathbf{M}_p$:

$$\dot{\mathbf{q}} = \mathbf{v}, \quad \mathbf{M}_p \dot{\mathbf{v}} = \mathbf{F}_p(\mathbf{q}, \mathbf{x}, \mathbf{R}) \quad (6.11)$$

Rigid Body System ($\mathbb{R}^3 \times \text{SO}(3)$):

Let $\mathbf{x} \in \mathbb{R}^3$ be the center of mass, $\mathbf{v}_{rb} \in \mathbb{R}^3$ the world-frame linear velocity, and M the mass. Let $\mathbf{R} \in \text{SO}(3)$ be the orientation, $\boldsymbol{\omega} \in \mathbb{R}^3$ the body-frame angular velocity, and $\mathbf{J} \in \mathbb{R}^{3 \times 3}$ the constant body-frame inertia tensor.

$$\dot{\mathbf{x}} = \mathbf{v}_{rb}, \quad M \dot{\mathbf{v}}_{rb} = \mathbf{F}_{rb}(\mathbf{q}, \mathbf{x}, \mathbf{R}) \quad (6.12)$$

$$\dot{\mathbf{R}} = \mathbf{R} \hat{\boldsymbol{\omega}}, \quad \mathbf{J} \dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times \mathbf{J} \boldsymbol{\omega} = \boldsymbol{\tau}_{body}(\mathbf{q}, \mathbf{x}, \mathbf{R}) \quad (6.13)$$

(Note: $\hat{\boldsymbol{\omega}} \in \mathfrak{so}(3)$ is the skew-symmetric matrix such that $\hat{\boldsymbol{\omega}} \mathbf{a} = \boldsymbol{\omega} \times \mathbf{a}$.)

6.5.2 The Staggered IMEX Integration Loop

To advance the state from t_n to t_{n+1} over interval h , we split the Velocity Verlet into two halves and nest the Implicit Midpoint Rule exactly between them.

Phase 1: Particle Explicit Velocity Half-Kick

Evaluate forces at the starting configuration t_n and advance particle velocities by a half-step:

$$\mathbf{v}_{n+1/2} = \mathbf{v}_n + \frac{h}{2} \mathbf{M}_p^{-1} \mathbf{F}_p(\mathbf{q}_n, \mathbf{x}_n, \mathbf{R}_n) \quad (6.14)$$

Phase 2: Particle Drift to Midpoint (Synchronization)

Advance the particle positions exactly to the integration midpoint $t_{n+1/2}$:

$$\mathbf{q}_{mid} = \mathbf{q}_n + \frac{h}{2} \mathbf{v}_{n+1/2} \quad (6.15)$$

Crucially, $\mathbf{q}_{mid}$ is now explicitly known. We freeze it to solve the implicit rigid body step.

Phase 3: Lie Group Implicit Solve (Rigid Body)

We define the rigid body midpoint velocities as the unknown variables $\mathbf{v}_{mid}$ and $\boldsymbol{\omega}_{mid}$. The

midpoint configuration is evaluated using linear translation and the $SO(3)$ Lie Group exponential map (Rodrigues' formula):

$$\mathbf{x}_{mid} = \mathbf{x}_n + \frac{h}{2}\mathbf{v}_{mid}, \quad \mathbf{R}_{mid} = \mathbf{R}_n \exp\left(\frac{h}{2}\hat{\boldsymbol{\omega}}_{mid}\right) \quad (6.16)$$

We substitute the discrete velocities $\dot{\mathbf{v}}_{rb} \approx \frac{2}{h}(\mathbf{v}_{mid} - \mathbf{v}_{rb,n})$ into the Newton-Euler equations evaluated at $t_{n+1/2}$. This yields a 6×6 non-linear root-finding problem $\mathbf{G}(\mathbf{v}_{mid}, \boldsymbol{\omega}_{mid}) = \mathbf{0}$:

$$\mathbf{G}_v = \frac{2M}{h}(\mathbf{v}_{mid} - \mathbf{v}_{rb,n}) - \mathbf{F}_{rb}(\mathbf{q}_{mid}, \mathbf{x}_{mid}, \mathbf{R}_{mid}) = \mathbf{0} \quad (6.17)$$

$$\mathbf{G}_\omega = \frac{2}{h}\mathbf{J}(\boldsymbol{\omega}_{mid} - \boldsymbol{\omega}_n) + \boldsymbol{\omega}_{mid} \times \mathbf{J}\boldsymbol{\omega}_{mid} - \boldsymbol{\tau}_{body}(\mathbf{q}_{mid}, \mathbf{x}_{mid}, \mathbf{R}_{mid}) = \mathbf{0} \quad (6.18)$$

(We solve this iteratively via Newton-Raphson. The exact geometric Jacobian is derived in Section 1.5.4).

Phase 4: Rigid Body Full State Update

Once Phase 3 converges to the exact roots $\mathbf{v}_{mid}^*$ and $\boldsymbol{\omega}_{mid}^*$, we definitively finalize the rigid body state at t_{n+1} :

$$\mathbf{x}_{n+1} = \mathbf{x}_n + h\mathbf{v}_{mid}^* \quad (6.19)$$

$$\mathbf{R}_{n+1} = \mathbf{R}_n \exp(h\hat{\boldsymbol{\omega}}_{mid}^*) \quad (6.20)$$

$$\mathbf{v}_{rb,n+1} = 2\mathbf{v}_{mid}^* - \mathbf{v}_{rb,n} \quad (6.21)$$

$$\boldsymbol{\omega}_{n+1} = 2\boldsymbol{\omega}_{mid}^* - \boldsymbol{\omega}_n \quad (6.22)$$

Phase 5: Particle Second Drift and Kick

Advance the particles to t_{n+1} and complete the Velocity Verlet loop:

$$\mathbf{q}_{n+1} = \mathbf{q}_{mid} + \frac{h}{2}\mathbf{v}_{n+1/2} \quad (6.23)$$

$$\mathbf{v}_{n+1} = \mathbf{v}_{n+1/2} + \frac{h}{2}\mathbf{M}_p^{-1}\mathbf{F}_p(\mathbf{q}_{n+1}, \mathbf{x}_{n+1}, \mathbf{R}_{n+1}) \quad (6.24)$$

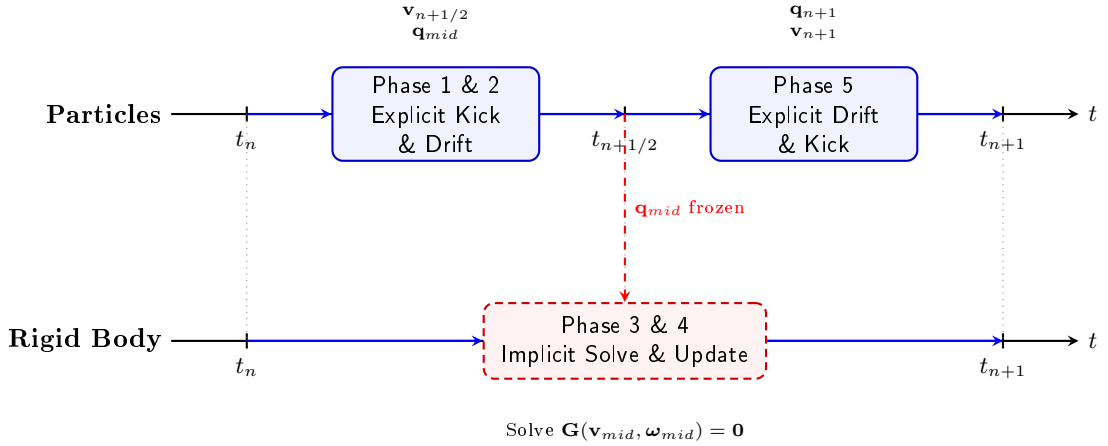


Figure 6.6. Partitioned Implicit-Explicit (IMEX) timeline. The particle system is explicitly stepped using a split Velocity Verlet scheme, which perfectly synchronizes the configuration exactly at the midpoint $t_{n+1/2}$. This intermediate state is locked to evaluate the computationally expensive implicit rigid body Newton-Euler solver before advancing both systems to t_{n+1} .

6.5.3 Proof of Exact Time-Reversibility

A numerical integrator is strictly time-reversible if starting from state S_{n+1} , negating the velocities, and taking a step of size $-h$ perfectly yields the original state S_n . This property bounds energy drift indefinitely.

Let's trace the algorithm backwards from t_{n+1} with step $-h$:

- **Reverse Phase 1:** $\mathbf{v}_{mid}^* = -\mathbf{v}_{n+1} + \frac{-h}{2}\mathbf{M}_p^{-1}\mathbf{F}_p(t_{n+1})$. Based on forward Phase 5, this explicitly evaluates to $-\mathbf{v}_{n+1/2}$.
- **Reverse Phase 2:** $\mathbf{q}_{mid}^* = \mathbf{q}_{n+1} + \frac{-h}{2}(-\mathbf{v}_{n+1/2}) = \mathbf{q}_{n+1} - \frac{h}{2}\mathbf{v}_{n+1/2} = \mathbf{q}_{mid}$.
- **Reverse Phase 3:** Because $\mathbf{q}_{mid}$ is identical, the implicit rigid body solver experiences the exact same environment. If we evaluate the residual using guesses $-\mathbf{v}_{mid}$ and $-\boldsymbol{\omega}_{mid}$, the backward midpoint geometries evaluate to:

$$\mathbf{x}_{mid}^* = \mathbf{x}_{n+1} + \frac{-h}{2}(-\mathbf{v}_{mid}) = \mathbf{x}_n + h\mathbf{v}_{mid} + \frac{h}{2}\mathbf{v}_{mid} = \mathbf{x}_{mid} \quad (6.25)$$

$$\mathbf{R}_{mid}^* = \mathbf{R}_{n+1} \exp\left(\frac{-h}{2}(-\hat{\boldsymbol{\omega}}_{mid})\right) = \mathbf{R}_n \exp(h\hat{\boldsymbol{\omega}}_{mid}) \exp\left(-\frac{h}{2}\hat{\boldsymbol{\omega}}_{mid}\right) = \mathbf{R}_{mid} \quad (6.26)$$

Since the configurations perfectly match, the coupling forces are identical. The inertial derivative $\frac{2}{-h}(-\mathbf{v}_{mid} - (-\mathbf{v}_{rb,n+1}))$ algebraically simplifies to exactly $\frac{2}{h}(\mathbf{v}_{mid} - \mathbf{v}_{rb,n})$. Thus, $\mathbf{G}^* = \mathbf{G} = \mathbf{0}$, meaning the exact negative mid-velocities are recovered.

- **Reverse Phases 4 & 5:** By symmetry, applying $-h$ with the converged roots perfectly recovers $\mathbf{x}_n, \mathbf{R}_n, \mathbf{v}_{rb,n}, \boldsymbol{\omega}_n$, and $\mathbf{v}_n$.

6.5.4 In-Depth Analytical Jacobian on Lie Group SO(3)

To solve Phase 3 in 2-3 iterations, we need the exact 6×6 Newton-Raphson block Jacobian $\mathcal{J} = \frac{\partial(\mathbf{G}_v, \mathbf{G}_\omega)}{\partial(\mathbf{v}_{mid}, \boldsymbol{\omega}_{mid})}$. Let the coupling force $\mathbf{F}_{world}$ apply at a constant body-frame attachment point $\mathbf{p}$. The body torque is $\boldsymbol{\tau}_{body} = \mathbf{p} \times (\mathbf{R}_{mid}^T \mathbf{F}_{world})$. Let $\mathbf{K} = \frac{\partial \mathbf{F}_{world}}{\partial \mathbf{x}_{world}}$ be the 3×3 translational stiffness matrix evaluated at the world point.

A. Lie Group Variations

If we perturb the midpoint angular velocity by $\delta\boldsymbol{\omega}_{mid}$, the right-trivialized tangent of the exponential map defines the variation in rotation $\delta\mathbf{R}_{mid}$:

$$\delta\mathbf{R}_{mid} = \mathbf{R}_{mid}\hat{\delta\boldsymbol{\phi}} \quad \text{where} \quad \delta\boldsymbol{\phi} = \frac{h}{2}\mathbf{J}_r\left(\frac{h}{2}\boldsymbol{\omega}_{mid}\right)\delta\boldsymbol{\omega}_{mid} \quad (6.27)$$

Here, $\mathbf{J}_r(\boldsymbol{\theta})$ is the Right Jacobian of SO(3):

$$\mathbf{J}_r(\boldsymbol{\theta}) = \mathbf{I} - \frac{1 - \cos\|\boldsymbol{\theta}\|}{\|\boldsymbol{\theta}\|^2}\hat{\boldsymbol{\theta}} + \frac{\|\boldsymbol{\theta}\| - \sin\|\boldsymbol{\theta}\|}{\|\boldsymbol{\theta}\|^3}\hat{\boldsymbol{\theta}}^2 \quad (6.28)$$

B. Force and Torque Derivatives

By tracking the variations of $\mathbf{x}_{mid}$ and $\mathbf{R}_{mid}$ induced by $\delta\mathbf{v}_{mid}$ and $\delta\boldsymbol{\omega}_{mid}$, we find the variation of the attachment point, and subsequently $\delta\mathbf{F}_{world}$ and $\delta\boldsymbol{\tau}_{body}$. Applying the chain rule and the identity $\hat{\mathbf{a}}\mathbf{b} = -\hat{\mathbf{b}}\mathbf{a}$, we extract the four blocks of the analytic Jacobian:

1. Translational Block (3×3):

$$\frac{\partial \mathbf{G}_v}{\partial \mathbf{v}_{mid}} = \frac{2M}{h}\mathbf{I} - \frac{h}{2}\mathbf{K} \quad (6.29)$$

2. Translational-Rotational Coupling Block (3×3):

$$\frac{\partial \mathbf{G}_v}{\partial \boldsymbol{\omega}_{mid}} = \frac{h}{2}\mathbf{K}\mathbf{R}_{mid}\hat{\mathbf{p}}\mathbf{J}_r \quad (6.30)$$

3. Rotational-Translational Coupling Block (3×3):

$$\frac{\partial \mathbf{G}_\omega}{\partial \mathbf{v}_{mid}} = -\frac{h}{2}\hat{\mathbf{p}}\mathbf{R}_{mid}^T\mathbf{K} \quad (6.31)$$

4. Rotational Block (3×3):

The rotational block contains the derivative of the Coriolis term $\frac{\partial}{\partial \boldsymbol{\omega}}(\boldsymbol{\omega} \times \mathbf{J}\boldsymbol{\omega}) = \widehat{\boldsymbol{\omega}}\mathbf{J} - \widehat{\mathbf{J}}\boldsymbol{\omega}$, combined with the Lie-mapped geometric stiffness:

$$\frac{\partial \mathbf{G}_{\boldsymbol{\omega}}}{\partial \boldsymbol{\omega}_{mid}} = \frac{2}{h}\mathbf{J} + \widehat{\boldsymbol{\omega}_{mid}}\mathbf{J} - \widehat{\mathbf{J}\boldsymbol{\omega}_{mid}} - \frac{h}{2} \left[\widehat{\mathbf{p}}(\mathbf{R}_{mid}^T \mathbf{F}_{world})^\wedge - \widehat{\mathbf{p}}\mathbf{R}_{mid}^T \mathbf{K} \mathbf{R}_{mid} \widehat{\mathbf{p}} \right] \mathbf{J}_r \quad (6.32)$$

(Where $\mathbf{J}_r \equiv \mathbf{J}_r(\frac{h}{2}\boldsymbol{\omega}_{mid})$). If multiple particles attach to the rigid body, you simply sum the stiffness contributions $\mathbf{K}_j$ and points $\mathbf{p}_j$ across these blocks.

Summary

By locking the explicit midpoint configuration $\mathbf{q}_{mid}$ during the implicit rigid body step, this IMEX formulation circumvents a massive $(3N+6) \times (3N+6)$ coupled Jacobian inversion. Instead, you only ever invert a dense 6×6 matrix, yielding a highly scalable algorithm that perfectly maintains $\mathcal{O}(h^2)$ precision, temporal symmetry, and exact adherence to the $\text{SO}(3)$ manifold.

Bibliography

- [1] P. J. Crutzen and E. F. Stoermer, «The 'anthropocene' (2000)», in *The Future of Nature*, New Haven: Yale University Press, 2013, pp. 479–490, ISBN: 9780300188479. DOI: [doi:10.12987/9780300188479-041](https://doi.org/10.12987/9780300188479-041). [Online]. Available: <https://doi.org/10.12987/9780300188479-041>.
- [2] T. Modis, «Forecasting the growth of complexity and change», *Technological Forecasting and Social Change*, vol. 69, no. 4, pp. 377–404, 2002, ISSN: 0040-1625. DOI: [https://doi.org/10.1016/S0040-1625\(01\)00172-X](https://doi.org/10.1016/S0040-1625(01)00172-X). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S004016250100172X>.
- [3] J. Huebner, «A possible declining trend for worldwide innovation», *Technological Forecasting and Social Change*, vol. 72, no. 8, pp. 980–986, 2005, ISSN: 0040-1625. DOI: <https://doi.org/10.1016/j.techfore.2005.01.003>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0040162505000235>.
- [4] H. P. Moravec, *Robot: Mere Machine to Transcendent Mind*. Oxford University Press USA, 1998.
- [5] R. Kurzweil, *Age of Spiritual Machines: When Computers Exceed Human Intelligence*, 1st. USA: Penguin Books, 1999, ISBN: 0140282025.
- [6] A. Quarteroni, F. Saleri, and P. Gervasio, *Calcolo scientifico: Esercizi e problemi risolti con MATLAB e Octave*. Springer Science & Business Media, 2012.